OCR Expense Tracker
Technical Report

Mini-Project 3 | Flutter 3.47.6 | Dart 3.13.5 | 2026-10-08

Student: [TODO: full name] | Student ID: [TODO] | Course/class: [TODO]

GitHub: [TODO] | APK: [TODO] | Demo video: [TODO]

1. Objective and required features

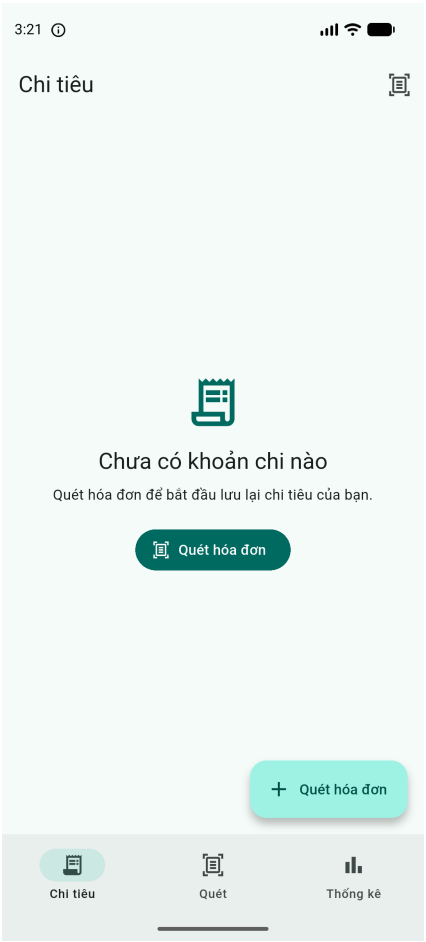
The Android app records expenses from receipt photos. It captures and crops an image, runs text recognition on-device, suggests merchant/date/total fields for correction, saves an expense locally, and displays history and statistics. The five fixed categories are Food, Study, Travel, Gear, and Entertainment.

Implemented flow: Camera and framing -> capture/crop -> Google ML Kit OCR -> heuristic parser -> editable review -> SQLite plus app-local receipt image -> Home history -> animated statistics. No backend, cloud OCR, account service, or third-party chart package is used.

2. Architecture and data flow

CameraScreen manages camera lifecycle, capture, crop, and OCR navigation. **OcrService** sends the local image path to ML Kit. **ReceiptParser** proposes fields from recognized text. Review validates edits and category before saving. **DatabaseService** stores expense fields; **ImageStorageService** copies the image to app documents storage and SQLite keeps its path. **ExpenseStatistics** feeds two CustomPainter charts.

Home / empty local history



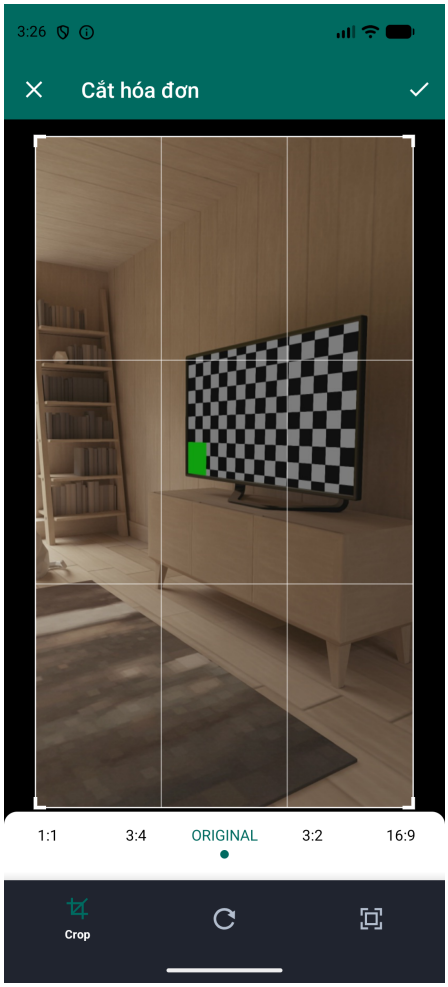
3. Camera, offline OCR, and heuristic parsing

The app selects the back camera when available, displays a receipt frame, supports flash where available and tap-to-focus, and releases/reopens the camera with app lifecycle changes. The captured file is cropped before OCR. Google ML Kit processes the cropped image locally; the app does not upload receipt data.

Parsing uses date patterns with slash or hyphen separators, Vietnamese total keywords, and grouped VND formats such as 150,000 VND and 150.000 ?. Total-related lines take priority over subtotal, discount, and tax. Suggestions can be missing or incorrect, so the review form permits manual correction.

Emulator camera preview (virtual scene) | Crop interface (same scene)





This emulator scene is not a receipt and did not demonstrate receipt recognition. Capture/crop reached the recoverable OCR error and manual-entry option.

4. Local storage and statistics

SQLite holds merchant, integer VND amount, transaction date, category, optional image path, and creation time. Receipt images are files under app documents storage. Statistics aggregates category totals and Monday-to-Sunday spending for the current week. Donut and bar charts are animated and painted directly with Flutter CustomPainter.

5. Verification results and limitations

Check	Result
dart format lib test	Pass; 25 files, no changes
flutter analyze	Pass; no issues
flutter test	Pass; 28 tests
Android debug APK	Pass
Android release APK	Pass; 81.3 MB; debug-signed test APK
Pixel_7 emulator	Release installed/launched; Home, preview, crop, empty Statistics rendered
Receipt OCR / persistence / data charts	Not verified with a real receipt or saved receipt expense

The emulator virtual scene produced an OCR error; the app showed manual recovery. A CameraX teardown PlatformException on the first initialization attempt was handled in app cleanup; a subsequent release retest showed live preview and crop without that unhandled error. A representative receipt was unavailable. Receipt field accuracy, save/history, thumbnail and relaunch persistence, and charts with saved data remain pending.

Empty Statistics on emulator (1080 x 2400)



Submission details

Student identity and final GitHub, APK-hosting, and demo URLs are TODO placeholders. The workspace has no Git repository or remote. The demo video has not been recorded. The APK uses the Android debug key; production distribution requires a private release keystore.